

Habu: A Checked Concatenative Kernel Target and Model CAD for GPUs

Joel Reymont
joelr1@gmail.com

Draft of July 4, 2026

Abstract

Writing custom GPU kernels remains an expert activity in which most errors surface only at runtime. We present Habu, a self-hosted checked Forth whose compiler types kernel compositions statically, and Maki, a model-development layer built on it that treats GPU compilation as computer-aided design: models are captured into an IR, fused by a planner with exact byte accounting and typed legality rules, lowered to generated PTX kernels, and *verified* — every generated kernel is proven against a checked host reference on the target device before its artifact is promoted. The same IR that executes also differentiates: adjoints are generated, gradient-checked, and used to train a model from scratch through the identical pipeline. The toolchain is a single self-hosted binary under 128 KB. On a Jetson Orin NX we measure parity with hand-written Triton at the streaming-memory roof and show that the checked target shifts the stack-discipline error class from runtime to author time. **[TODO: Revise abstract once compute-bound results land (§7).]**

1 Introduction

The deep learning compilation landscape divides labor in a familiar way: vendor libraries cover common operations; kernel DSLs such as Triton [1] let experts write the rest; graph compilers fuse and schedule above them. Two costs are accepted as normal. First, kernel authorship is unchecked: a Triton author who mis-tracks a mask or an offset discovers it at runtime, if at all. Second, the artifacts of compilation are *emitted, not proven*: nothing in the toolchain certifies that the fused kernel computes what the unfused graph did.

Habu rejects both costs. It is a Forth in the lineage of Chuck Moore’s OKAD [2]: a small, self-hosted system (the complete engine, compiler, and checker fit in a binary under 128 KB) in which programs are compositions of words, and—the departure from tradition—every word carries a typed, row-polymorphic stack effect that the self-hosted checker verifies. Kernels for the GPU are written (or generated) in the same checked language and emitted as PTX text.

Maki, the model layer, treats the remaining pipeline as computer-aided design rather than black-box compilation. A model is captured into an IR by a checked definition (`MODEL:`); a fusion planner groups nodes into regions under typed legality rules, reporting exact traffic in bytes; a memory planner reports coalescing classes; a schedule layer names candidate kernels per region family; and a lowering layer generates one PTX kernel per region. At every stage the system emits facts a person (or an agent) can audit, and the final artifact is admitted only through gates:

- **CERTIFY** — static legality of the whole plan;

- **GOLDEN** — the generated device kernels reproduce a checked host executor’s results on the target hardware, element-wise under per-class tolerances (or match an external reference artifact);
- **GRADCHECK** — generated adjoints agree with central finite differences;
- **PROFILE** — measured occupancy and roofline placement (mandatory to run, non-blocking).

Only a model that passes is *promoted*: its schedule selection and evidence are recorded in a content-addressed store keyed by region signature, shape class, dtype, layout, alignment, target, and engine build.

This paper contributes: (1) the design of a checked concatenative kernel target and the measurement that it shifts a whole error class to author time (§6); (2) Model CAD — a verification-first compilation pipeline whose every generated kernel is device-proven against a checked reference (§4); (3) a verified-gradient facility in which the training IR and the gradchecked IR are the same object, demonstrated by training a model from scratch through the pipeline (§5); and (4) measurements on Jetson Orin NX (§??). **[TODO: Add contribution (5): compute-bound performance results, once measured.]**

2 Background and Related Work

Tillet et al.’s Triton paper [1] classifies prior systems as tensor-level IRs (XLA, Glow: pattern-matched templates), polyhedral compilers (Tensor Comprehensions, Diesel: affine loop nests), and loop synthesizers (Halide, TVM: user-specified schedules), positioning Triton as tile-level operations inside a traditional pipeline with compiler-owned parallelization. Habu shares Triton’s durable insight — the author writes a scalar-looking program over tiles; the compiler owns intra-tile concurrency — but differs on three axes. First, the composition is *typed*: fused chains are checked words whose stack effects the compiler verifies, so a malformed composition fails to compile rather than misbehaving on device. Second, fusion is a *planner decision with an audit trail* (regions, split reasons, byte deltas), not an authoring convention. Third, artifacts are *gated*: device-vs-host verification is part of compilation, not an external test the author may skip. Modern Triton (Python DSL on MLIR) retains per-kernel horizons and per-process JIT autotuning; Habu’s tuning is persistent and content-keyed. **[TODO: 1–2 paragraphs comparing against torch.compile/Inductor as the automatic-fusion baseline, with citations.]**

3 The Checked Kernel Target

[TODO: Expand: Habu’s type system (row-polymorphic stack effects, nominal roles), the PTX emit vocabulary (tile words, collective words, module convention: one header, many entries), a worked fused-kernel listing mirroring docs/eval-triton.md, and the checker-miss RCA discipline. Source material: docs/forth.md, lib/ptx/*, docs/eval-triton.md.]

3.1 Type Families

[TODO: BLOCKED on the type-families campaign merging (TFAM 9/10/12/14/15) and the cad-adt-swap adoption. When landed, write: checker-enforced algebraic data types in a self-hosted Forth (sum types with MATCH dispatch and bad-tag proofs,

enum and product families, packed ABI vs boxed policy); then the adoption evidence per the ablation rule — the CAD internals (schedule key as a product record with enum fields, gate verdicts as sums) where the checker turns field-swap and tag-confusion bugs into compile errors; cite the negative regressions. Do not write this section from design docs alone; it requires the merged implementation and its gates.]

4 Model CAD

[TODO: Expand from CAD-PLAN.md and docs/model-cad.md: capture (MODEL: with named value references and true skip connections), typed shape legality at capture, fusion planner + exact byte accounting (FFN example: 3040→2272 bytes, 1.33×), memory planning, schedule families and the §7.4 content key, the four lowering classes (elementwise, row-reduce, matmul, movement) each generating PTX from the region, and the gate set with promote semantics. Include the generated REGION kernel PTX excerpt for a fused GELU→RELU region.]

5 Verified Gradients and Training

[TODO: Expand: adjoint registry, reverse transform with fan-out summation, save-vs-recompute, tensor-granularity gradcheck against the host executor, and the from-scratch flagship: seeded windowed-MLP regression trained 60 SGD steps, batch NLL 0.130→−0.647, bit-deterministic, the same IR gradchecked before training. Source: maki/from-scratch-*.f, maki/backward.f, maki/gradcheck.f.]

6 Error-Class Measurement

On identical tasks against real Triton 3.5.1 on the same device, a six-candidate error battery shows the checked target catching stack-discipline and composition errors at author time with located diagnostics and zero GPU involvement, where Triton surfaces them at runtime or not at all; authoring studies (pass@1 = 3/3 both targets on a fused chain) show reachability is comparable, so the difference is *where bugs are caught*, not what experts can express. [TODO: Import the full battery table and pass@k protocol from docs/eval-triton.md; extend candidates and task classes per the ablation matrix in the paper dot.]

7 Performance

Measured today. Memory-bound: SAXPY streaming at 63 GB/s equals hand-written Triton (63.0/63.4 GB/s) at this Orin NX's achievable read/read/write bandwidth; the fused `relu(a·x+y)` chain reaches the same roof, produced automatically from checked words. Generated-kernel correctness: 22 models device-golden across the four region classes. [TODO: The compute-bound section is intentionally empty until measured (CAD-PLAN §8.1): on-device roofline; GEMM vs Triton baseline; cp.async stages; tensor-core MMA; persistent autotune vs JIT warmup; end-to-end model latency vs torch.compile. Do not write prose here — insert the measurement tables when they exist.]

8 Effectiveness of Each Mechanism

[**TODO: The ablation matrix (one committed experiment per invention) from dot habu-paper-habu-checked: checker error battery; byte-model predicted-vs-measured; fusion ON/OFF; golden fault-injection; sentinel fault-injection; wrong-adjoint detection; persistent-tune warmup comparison; EXPLAIN-packet repair-loop metrics; tuned-vs-default deltas. Each row cites its in-tree gate. Run on device; report tables.**]

9 Limitations

V1 caps are explicit and fail closed: one contraction per region; $K \leq 256$; elementwise prologues into contractions are planned but not yet lowered; broadcast operands and multi-output regions are staged for the OPTIMIZE wiring; PROFILE’s roofs await device microbenchmarks. The checker types stack discipline and roles, not semantics — numeric correctness is the golden gate’s job, by design. [**TODO: Refresh against the dot state at submission time.**]

10 Conclusion

[**TODO: Write last.**]

References

- [1] P. Tillet, H. T. Kung, D. Cox. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. MAPL 2019.
- [2] C. Moore. OKAD: VLSI design tools in colorForth. [**TODO: proper citation**]
[**TODO: Add: Halide, TVM, PlaidML, Tensor Comprehensions, XLA, roofline, torch.compile/Inductor.**]